

LC 1466: Reorder Routes to Make All Paths Lead to City Zero

Approach Discussion

Basically $\rightarrow$ whenever there is a connection from `curr  $\rightarrow$  next` we need to reverse and do `next  $\rightarrow$  curr` with `curr` starting from 0. Each such operation does `ans++`.

Example Visualization

Consider the structure: $0 \leftarrow 1 \rightarrow 2 \leftarrow 3 \rightarrow 4$



In the above graph, the connections `1  $\rightarrow$  2` and `3  $\rightarrow$  4` are pointing away from city 0.

These are invalid connections, so `ans = 2`.

Adjacency List Construction

So in the adjacency list we push:

```
[1, 0]  $\rightarrow$  1  $\rightarrow$  0, 1  
0  $\rightarrow$  1, 1
```

This means 1 and 0 are neighbours with 1 as orig start.

C++ Code Implementation

Below is the complete code for the discussed approach. Spacing, logic, and indentation have been preserved exactly as authored.

```
class Solution {
public:
    int minReorder(int n, vector<vector<int>>& connections) {

        vector<vector<pair<int, int>>> adj(n);
        int x = connections.size();
        vector<int> seen(n, 0);

        for (int i = 0; i < x; i++) {
            int u = connections[i][0];
            int v = connections[i][1];

            adj[u].push_back({v, u});
            adj[v].push_back({u, u});

            // adjacnecy is storing u->v with original start from u
        }

        int curr = 0;
        int ans = 0;

        for (int i = 0; i < n; i++) {
            seen[curr] = 1;
            int next;
            int origst;

            for (auto& it : adj[curr]) {

                next = it.first;
                origst = it.second;

                if (seen[next])
                    continue;

                if (origst != curr) {
                    // 0->1 but start was from 1
                    // so 1->0 which is valid
                    // do nothing cause itss valid
                    ans = ans + 0;
                }

                else {
                    ans++; }
            }

            curr = next;
        }

        return ans;
    }
};
```

```
seen[next]=1;
```

```
q.push(next);
```

```
}
```